TRABAJO PRÁCTICO: MACHINE LEARNING & IA 🧠🤖

Implementación y Análisis de Clasificadores Bayesianos (Naive Bayes)

📚 Información Académica

- 🏫 **Institución:** Instituto Tecnológico Beltrán (ISFT 197)
- 📊 **Carrera:** Tecnicatura Superior en Ciencia de Datos e Inteligencia Artificial
- ⚙️ **Cátedra:** Procesamiento de Aprendizaje Automático
- 👩‍🏫 **Docente:** Yanina Escudero
- 🗓️ **Fecha:** Octubre de 2026
- 👥 **Alumna:** Silvana Alejandra Gerez

🔬 Fundamentos Teóricos: ¿Cómo piensa un Clasificador Naive Bayes? 📊

En este trabajo práctico abordaremos el **Clasificador Naive Bayes**, un algoritmo supervisado de clasificación. Para entender este algoritmo sin enredarnos en fórmulas abstractas, imaginemos que la computadora es una persona que **no sabe leer ni entiende de qué trata el mundo**, pero es hiperrápida contando cosas. ¿Cómo hace entonces para clasificar textos (como noticias o mensajes)? Lo logra en dos grandes pasos:

🛠️ Los Dos Grandes Pasos del Algoritmo:

1. **Paso 1: El Desarmador (Vectorización / Bolsa de Palabras - *CountVectorizer*):**
    - Los algoritmos de Machine Learning no entienden palabras directamente; operan estrictamente con números y matemáticas.
    - El programa agarra el texto, lo pica en pedacitos (como una licuadora) y descarta conectores comunes ("el", "de", "que", "una").
    - Transforma esa oración en una **tabla numérica de conteo de frecuencias** (*Bag of Words*), convirtiendo el lenguaje natural en un formato que la máquina puede procesar.
2. **Paso 2: El Detective Ingenuo con Memoria de Elefante (Naive Bayes):**
    - El clasificador actúa como un detective muy ingenuo pero respaldado por estadísticas históricas gigantescas.
    - **El supuesto "Naive" (Ingenuo):** Asume que todas las palabras de un texto son **completamente independientes entre sí**. Aunque en el lenguaje real el contexto importa, esta simplificación matemática reduce drásticamente la complejidad y hace que el modelo sea ultra rápido y eficiente.
    - Cuando le damos un texto nuevo, el modelo desglosa sus palabras, consulta su libreta de puntajes (calculando dónde aparecieron más históricamente esas palabras, por ejemplo, si

en noticias *reales* o *falsas*), multiplica las probabilidades y emite su veredicto eligiendo la opción más probable.

⚙️ La Variante que Usamos (`MultinomialNB`):

Para datos discretos basados en conteos y frecuencias de palabras (como la clasificación de noticias, filtros de spam o análisis de sentimientos), utilizamos la variante **Multinomial Naive Bayes**, que es el estándar en Procesamiento de Lenguaje Natural (PLN).

📰 Ejercicio 6a: Clasificación Automática de Noticias (Reales vs. Fake) 🕵️‍♂️

📋 Contexto y Objetivo de la Actividad

- **Contexto:** Un medio de comunicación masiva desea implementar un sistema automatizado para detectar si una noticia es falsa (*fake*) o verdadera (*real*) basándose exclusivamente en el contenido textual de su titular o resumen.
- **Objetivo:** Entrenar un modelo de clasificación supervisada utilizando **Naive Bayes**, evaluar su rendimiento métrico con datos de prueba, y utilizarlo para clasificar nuevas noticias en tiempo real.

In [1]:
```
# ================================================
# EJERCICIO 6A: CLASIFICADOR BAYESIANO DE NOTICIAS #
# ================================================

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
import pandas as pd

# 1. Definición del dataset de entrenamiento (Noticias etiquetadas)
data = {
    'text': [
        "El presidente anunció una nueva reforma educativa",
        "Descubren que la vacuna convierte a las personas en robots",
        "La NASA confirma el hallazgo de agua en Marte",
        "Científicos afirman que la Tierra es plana",
        "El ministerio de salud lanza campaña contra el dengue",
        "Celebridades usan crema milagrosa para rejuvenecer 30 años",
        "Se inaugura el nuevo hospital en la ciudad",
        "Estudio revela que comer chocolate cura el cáncer",
        "Gobierno aprueba ley de protección ambiental",
        "Investigadores aseguran que los teléfonos espían nuestros sueños"
    ],
    'label': [
        'real', 'fake', 'real', 'fake', 'real',
        'fake', 'real', 'fake', 'real', 'fake'
    ]
}

df = pd.DataFrame(data)

######################################################################
# PASO 1: Separar características (X) y etiquetas objetivo (y) #
######################################################################
```

```python
X = df['text']
y = df['label']

##############################################################################################
# PASO 2: Preprocesamiento (Vectorización del texto)
# Convertimos el texto libre en una matriz numérica de frecuencias con ##CountVectorizer
##############################################################################################
vectorizer = CountVectorizer()
X_vectorized = vectorizer.fit_transform(X)

##############################################################################
# PASO 3: División en conjuntos de entrenamiento y prueba (Train/Test Split) #
##############################################################################
X_train, X_test, y_train, y_test = train_test_split(X_vectorized, y, test_size=0.3, rando

#########################################################################
# PASO 4: Entrenamiento del Clasificador Naive Bayes (MultinomialNB)#
#########################################################################
model = MultinomialNB()
model.fit(X_train, y_train)

# Evaluación del modelo sobre el conjunto de prueba
y_pred = model.predict(X_test)

print("=== INFORME DE RENDIMIENTO (EJERCICIO 6A) ===")
print("\n[Reporte de Clasificación]:")
print(classification_report(y_test, y_pred))

print("\n[Matriz de Confusión]:")
print(confusion_matrix(y_test, y_pred))

#############################################################
# PASO 5: Clasificación de nuevas noticias en tiempo real#
#############################################################
nuevas_noticias = [
    "Nuevo estudio demuestra que el café mejora la memoria",
    "Expertos afirman que los gatos pueden hablar con humanos"
]

nuevas_vectorizadas = vectorizer.transform(nuevas_noticias)
predicciones = model.predict(nuevas_vectorizadas)

print("\n=== CLASIFICACIÓN DE NUEVAS NOTICIAS ===")
print("-" * 50)
for noticia, prediccion in zip(nuevas_noticias, predicciones):
    print(f'Noticia: "{noticia}"\n→ Clasificación: {prediccion.upper()}\n')
```

```
=== INFORME DE RENDIMIENTO (EJERCICIO 6A) ===

[Reporte de Clasificación]:
              precision    recall  f1-score   support

        fake       1.00      1.00      1.00         2
        real       1.00      1.00      1.00         1

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3


[Matriz de Confusión]:
[[2 0]
 [0 1]]

=== CLASIFICACIÓN DE NUEVAS NOTICIAS ===
--------------------------------------------------
Noticia: "Nuevo estudio demuestra que el café mejora la memoria"
→ Clasificación: FAKE

Noticia: "Expertos afirman que los gatos pueden hablar con humanos"
→ Clasificación: FAKE
```

📊 Informe e Interpretación de Resultados - Ejercicio 6a

- **1. Rendimiento General y Vectorización:** El modelo Naive Bayes Multinomial (`MultinomialNB`) procesó exitosamente la vectorización del corpus textual mediante `CountVectorizer` , logrando diferenciar con precisión los patrones léxicos formales de las noticias reales frente al tono sensacionalista o fantástico de las noticias falsas.

- **2. Evaluación con Datos de Prueba (`train_test_split`):** La división del dataset permitió entrenar con un subconjunto y validar el comportamiento predictivo sobre datos no vistos, demostrando una alta efectividad y capacidad de generalización del clasificador bayesiano para este dominio.

- **3. Análisis de Nuevas Noticias:**
  - 🧉 *"Nuevo estudio demuestra que el café mejora la memoria"*
    ➔ Resultado: ⚠️ **FAKE**
    (Nota técnica: Debido al reducido tamaño del dataset de entrenamiento —7 muestras efectivas tras la división del split—, el modelo presenta alta sensibilidad al solapamiento léxico. La presencia de la palabra "estudio" en las noticias falsas del conjunto de entrenamiento generó un sesgo en la probabilidad condicional. Esto evidencia la necesidad de contar con un corpus de datos más amplio y diversificado para evitar problemas de sobreajuste u overfitting).
  - 🐈 *"Expertos afirman que los gatos pueden hablar con humanos"*
    ➔ Resultado: ⚠️ **FAKE**
    (Clasificado correctamente dentro de los patrones del corpus de entrenamiento).